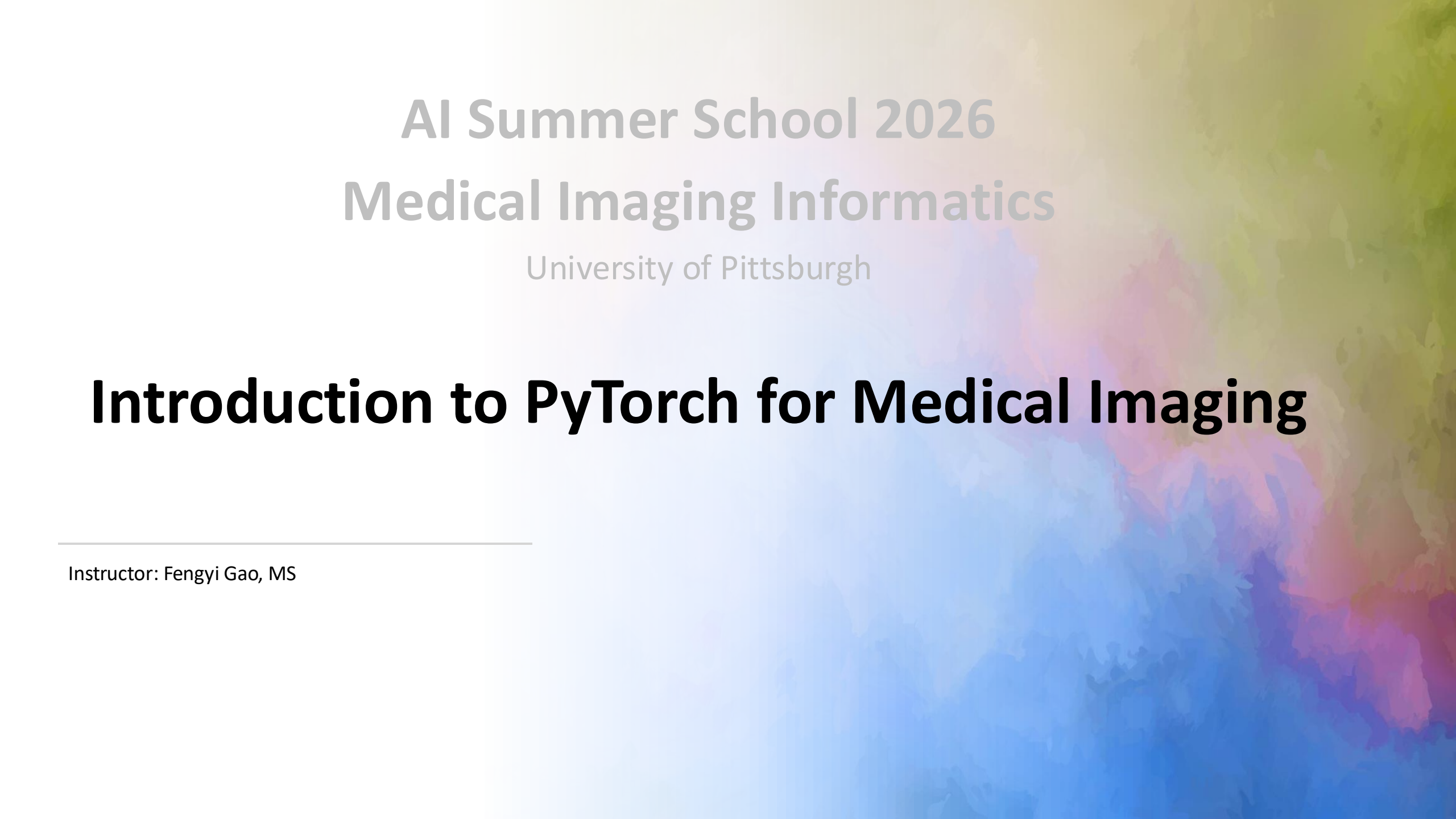




AI Summer School 2026

Medical Imaging Informatics

University of Pittsburgh

Introduction to PyTorch for Medical Imaging

Instructor: Fengyi Gao, MS

Learning Objectives

After completing this lecture, you should be able to:

- Explain what PyTorch is and why it is used
- Understand what a tensor is and its properties
- Understand how to work with data in PyTorch using Datasets and DataLoaders
- Explain why GPUs are an important component of deep learning
- Understand the basic workflow for training and evaluating a neural network in PyTorch

Outline

- PyTorch: What and Why?
- Tensors
- Working with Data
- nn Module
- GPUs
- Training and Evaluating a Neural Network
- Applications

PyTorch: What and Why?

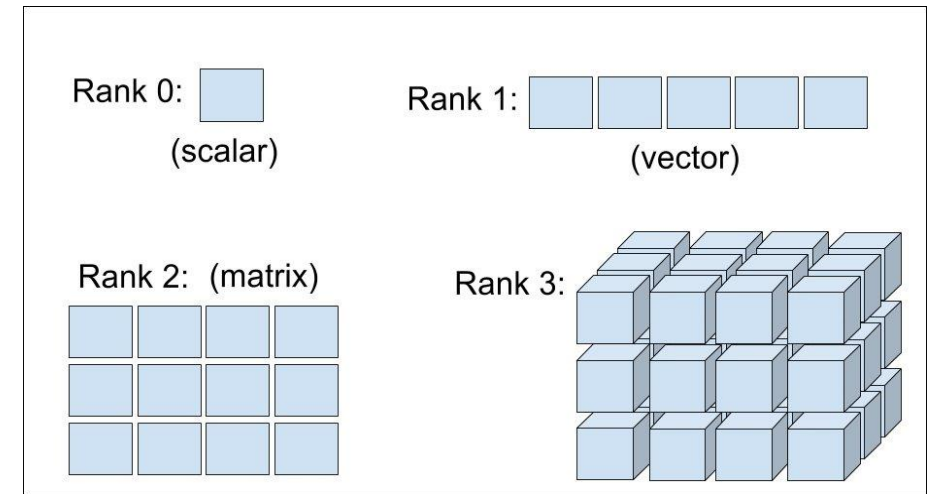
- **What is it?** An open-source machine learning framework used to build and train neural networks.
- **Who made it?** Facebook's AI Research Lab (FAIR) — now used by universities and companies worldwide
- **Why use it?** Friendly Python code, works with NumPy, provides automatic differentiation for training deep learning models
- **Applications:**
 - Medical imaging
 - Autonomous vehicles
 - Face recognition
 - Language translation
 - Generative AI
- **Install it:** `pip install torch torchvision torchaudio`



Source: PyTorch Tutorials (PyTorch, n.d.)

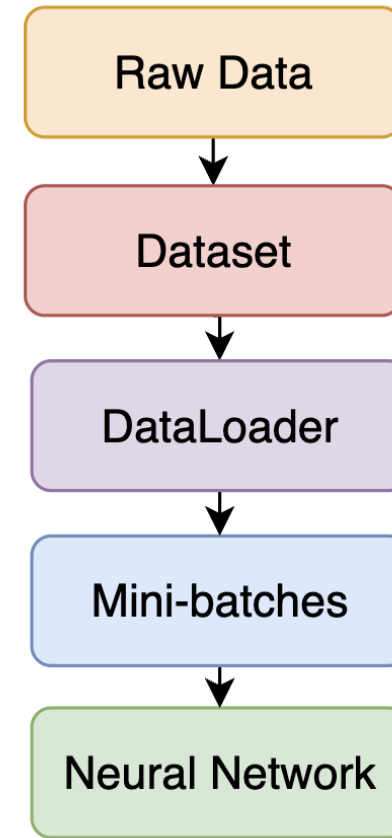
Tensors: The Data Structure of Deep Learning

- Tensors are the primary data structure in PyTorch. They store numerical data and support efficient computation on CPUs and GPUs.
- Tensors can represent data with multiple dimensions, such as vectors, matrices, images, and videos.
- In PyTorch, all inputs, outputs, and model parameters are stored as tensors.
- Every tensor has 3 key properties:
 - **Shape:** the size of the tensor
 - **Data type:** type of data stored in the tensor
 - **Device:** the device where the tensor is stored (CPU or GPU)



Working with Data

- PyTorch provides tools for organizing and loading data efficiently during training and evaluation.
- **Dataset**
 - Stores and organizes data samples
 - Provides access to individual images and labels
 - Acts as the bridge between raw data and the model
- **DataLoader**
 - Loads data efficiently during training
 - Groups samples into mini-batches
 - Can shuffle data to improve learning



Working with Data: Datasets

- **What Does a Dataset Do?**
- A Dataset defines:
 - The total number of samples available
 - How individual samples are accessed
 - The information returned to the model (e.g., image and label)
- In PyTorch, custom datasets are typically created by inheriting from the Dataset class and implementing two key methods:
 - `__len__()`
 - returns the number of samples
 - `__getitem__()`
 - returns a specific sample

```
from torch.utils.data import Dataset

class MedicalDataset(Dataset):

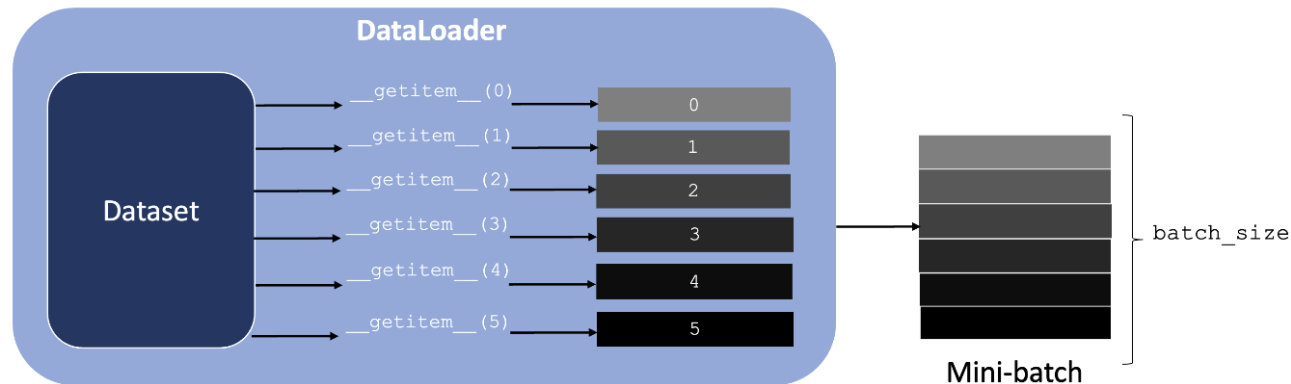
    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        return self.images[idx], self.labels[idx]
```

Dataset = "How many samples do I have?" + "How do I get one sample?"

Working with Data: Dataloaders

- **Why Use a DataLoader?**
 - Training often involves thousands of images.
 - Loading the entire dataset at once may be inefficient or exceed available memory.
- **DataLoader Responsibilities**
 - Loads data automatically
 - Creates mini-batches
 - Shuffles data when needed
 - Improves training efficiency
- **Key Parameters**
 - `batch_size`: Number of samples processed together
 - `Shuffle`: Randomizes data order before each training epoch



nn Module: Defining Neural Networks

- nn.Module is the foundation for building neural networks in PyTorch.
- Provides building blocks such as layers, activation functions, and loss functions.
- We define the network architecture and the `forward()` function that describes how data flows through the model.

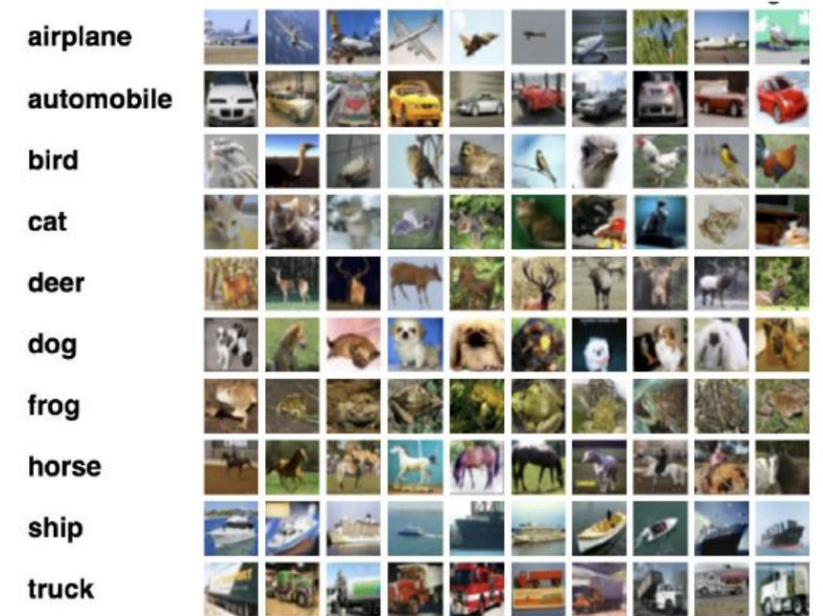
```
class SimpleClassifier(nn.Module):  
  
    def __init__(self, num_inputs, num_hidden, num_outputs):  
        super().__init__() # Initialize the modules we need to build the network  
        self.linear1 = nn.Linear(num_inputs, num_hidden)  
        self.act_fn = nn.Tanh()  
        self.linear2 = nn.Linear(num_hidden, num_outputs)  
  
    def forward(self, x):  
        # Perform the calculation of the model to determine the prediction  
        x = self.linear1(x)  
        x = self.act_fn(x)  
        x = self.linear2(x)  
        return x
```

```
model = SimpleClassifier(num_inputs=2, num_hidden=4, num_outputs=1)  
# Printing a module shows all its submodules  
print(model)
```

```
SimpleClassifier(  
  (linear1): Linear(in_features=2, out_features=4, bias=True)  
  (act_fn): Tanh()  
  (linear2): Linear(in_features=4, out_features=1, bias=True)  
)
```

torchvision

- torchvision is a PyTorch package designed for computer vision tasks.
- **Datasets**
 - Ready-to-use datasets for learning and experimentation
 - Examples: CIFAR-10, MNIST, ImageNet
- **Image Transformations**
 - Common preprocessing operations
 - Examples: Resize, Crop, Normalize, Data Augmentation
- **Pretrained Models**
 - Models already trained on large image datasets
 - Examples: ResNet, DenseNet, Vision Transformer (ViT)
- **Why Use torchvision?**
 - Reduces development time
 - Provides reliable implementations
 - Makes transfer learning easier



Source: My Learnings in AI/ML (n.d.)

GPUs

- Training neural networks requires substantial computational resources and memory.
- **Graphics Processing Units (GPUs)** can speed up computations and provide the memory needed
- PyTorch provides ways to transfer data from the CPU to a GPU.
- GPUs can execute many computations in parallel, making deep learning training significantly faster than CPUs.
- Steps:
 - Determine whether a GPU is available and get the device name:
`torch.cuda.is_available()`
 - Transfer model to GPU
 - During training transfer the data to the GPU

```
# Set device for training
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
# Move model to device
model = model.to(device)
```

GPUs

- GPUs are available in Google Colab
- In a Colab session:
 - Select **Runtime > Change Runtime Type > T4 GPU > Save**

Change runtime type

Runtime type

Python 3 ▼

Hardware accelerator [?](#)

☐ CPU ☒ T4 GPU ☐ A100 GPU ☐ L4 GPU
☐ TPU (deprecated) ☐ TPU v2

Want access to premium GPUs? [Purchase additional compute units](#)

Cancel

Save

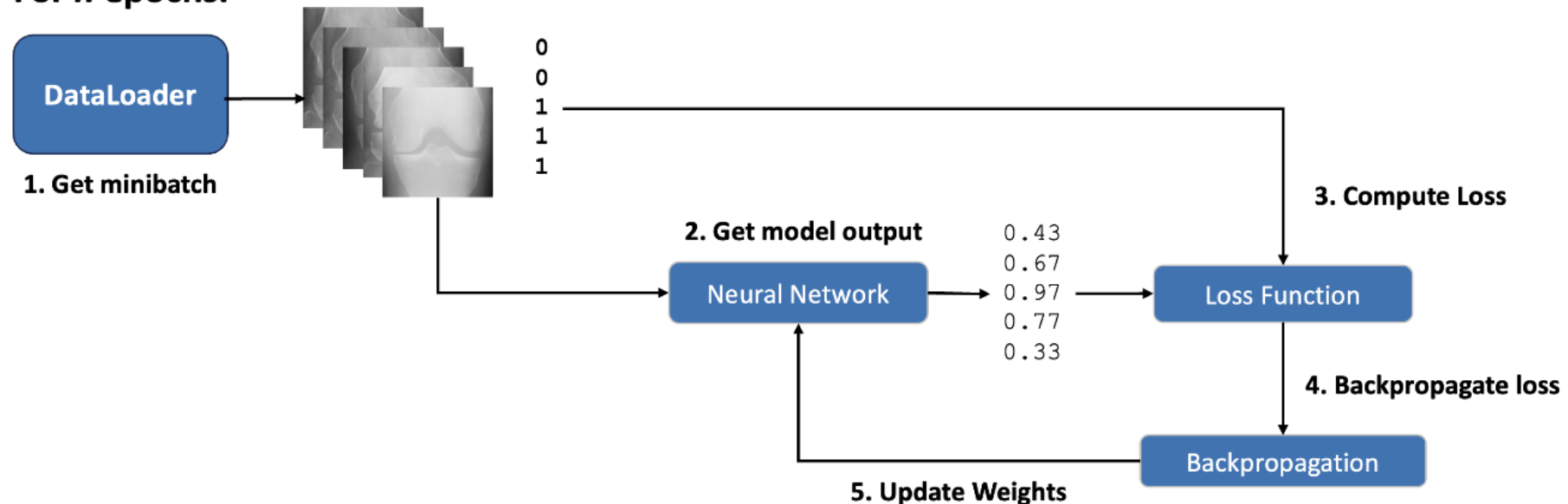
Training Neural Networks: Key Components

- **Model**
 - Learns patterns from data
 - Produces predictions from input images
- **Loss Function**
 - Measures the difference between predictions and ground truth
 - Provides feedback on model performance
 - Lower loss generally indicates better predictions
- **Optimizer**
 - Updates model parameters during training
 - Helps the model improve over time
- **Learning Rate**
 - Controls how much model parameters change during each update
 - Too small → slow learning
 - Too large → unstable learning
- **Epoch**
 - One complete pass through the training dataset
 - Models usually require multiple epochs to learn effectively

Training Neural Networks: Steps

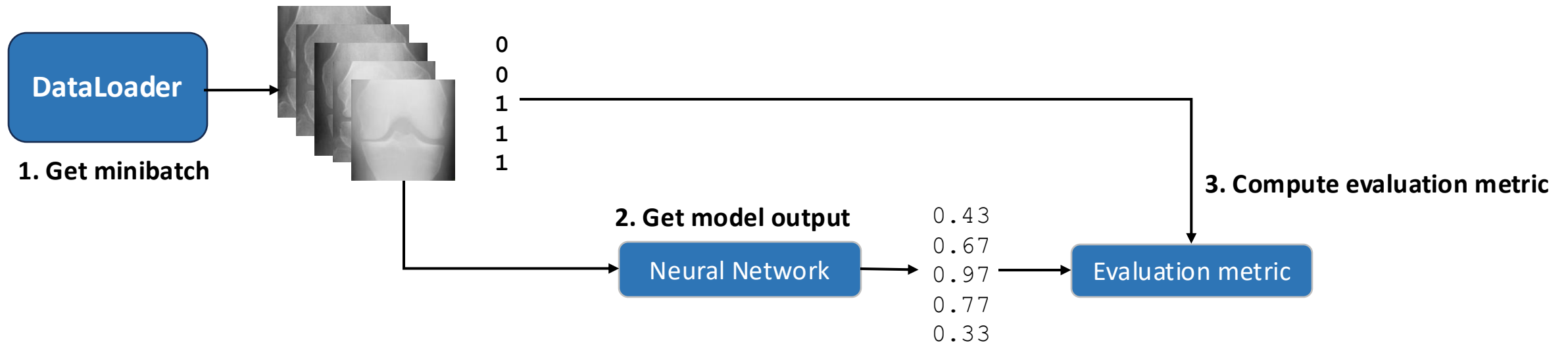
1. Get training batch from the data loader
2. Obtain predictions from the model for the batch
3. Calculate the loss between the predictions and the actual labels
4. Backpropagation
5. Update the model parameters
6. Evaluate on validation (if available)
7. The training process is repeated over many epochs until the model performance stabilizes.

For n epochs:



Evaluating a Neural Networks: Validation Steps

1. Get a batch of test data (unseen by the model)
2. Get the predictions from the model
3. Measure performance using metrics such as Accuracy, F1-score, Precision, Recall, and IoU



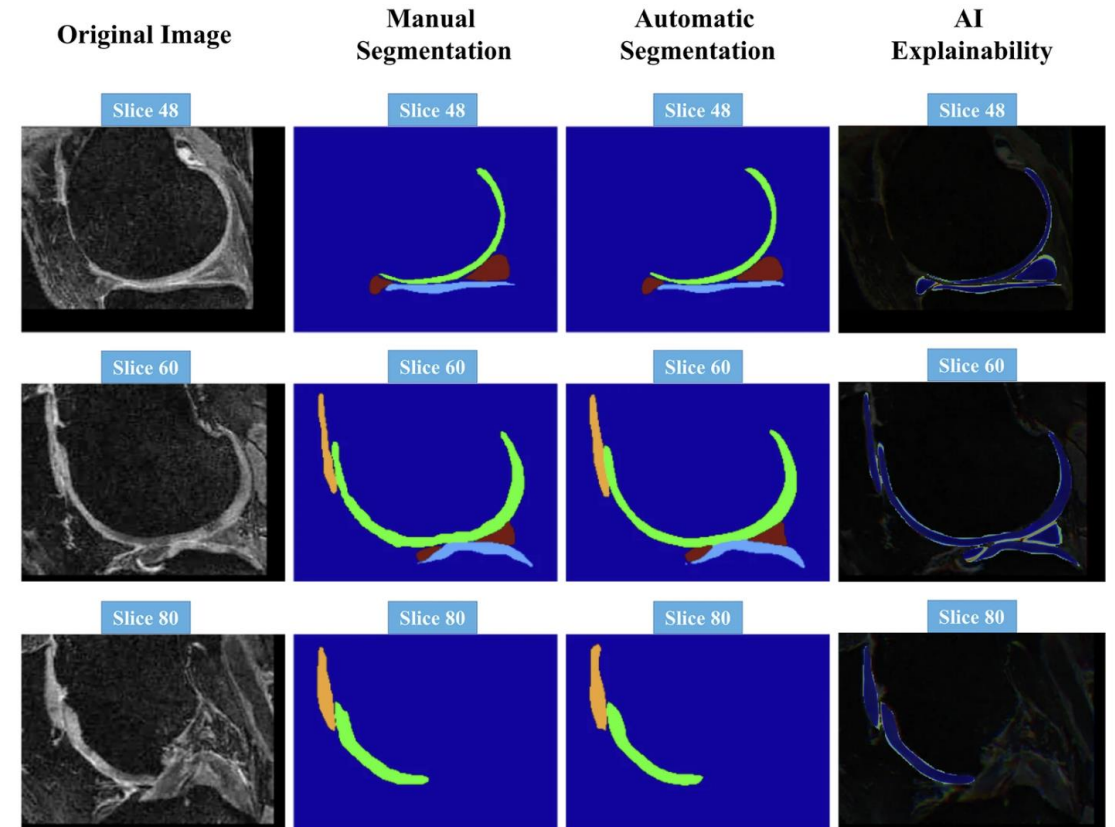
Key Takeaway: Evaluation is performed on data that was not used during training to estimate how well the model generalizes to new samples.

Overfitting vs. Underfitting: Why Validation Matters

- **Overfitting:** Model performs well on training data but poorly on unseen data.
 - Fix: use dropout layers, regularization (L1/L2), or get more training data
- **Underfitting:** The model is too simple to learn meaningful patterns from the data.
 - Fix: add more layers or neurons, train longer, or provide more/better features
- **Golden Rule:** Train set = learning, Validation set = tuning, Test set = final grade. Never peek at test data during training!
- **Evaluation Metrics:** Accuracy, F1-Score, Precision, Recall, IoU — always test on data the model has NEVER seen!
- **The goal is to achieve good performance on both training and unseen data.**

Applications of PyTorch in Medical Imaging

- Disease classification from chest X-rays
- Breast cancer detection from mammography
- Brain tumor segmentation from MRI scans
- Organ detection in CT images
- Pathology whole-slide image analysis
- Osteoarthritis grading from knee radiographs



Thank you!

Questions!



GitHub



Pitt AI Summer School